

PHP avancé : structures de contrôle et interfaçage avec MySQL

Module 1 : Programmation

Durée : 2h. Documents autorisés : cours, TP précédents. Les réponses justifiées sont attendues. Le code PHP doit être syntaxiquement correct.

EXERCICE 1 Gestion des inscriptions – Structures de contrôle

L'association `Jeunes_Entrepreneurs_CM` organise un concours à Yaoundé. Les frais d'inscription varient selon l'âge et la catégorie professionnelle. Le tableau ci-dessous résume la grille tarifaire.

Catégorie	Âge	Tarif (FCFA)
Étudiant	< 25 ans	5 000
Étudiant	≥ 25 ans	7 500
Professionnel	< 30 ans	15 000
Professionnel	≥ 30 ans	20 000
Autre	–	10 000

On dispose des variables PHP `$age` (entier), `$categorie` (chaîne : "etudiant", "professionnel", "autre") et `$estMembre` (booléen). Les membres bénéficient d'une réduction de 10% sur le tarif de base.

1. Écrire une structure conditionnelle (if/elseif/else) qui affecte à `$tarifBase` le montant correspondant selon le tableau.
2. Compléter le script pour calculer `$tarifFinal` en appliquant la réduction si `$estMembre` est vrai. Arrondir à l'entier supérieur (`ceil`).
3. En utilisant une boucle `for`, afficher un récapitulatif pour les âges de 20 à 35 ans (pas de 5) pour un étudiant non membre. Format : Âge : 20 -- Tarif : 5000 FCFA.
4. Justifier en une phrase le choix de `elseif` plutôt que de `switch` pour la question 1.

EXERCICE 2 Connexion et création de table – Extension mysqli

On souhaite créer une base de données `concours_yaounde` et une table `inscrits` sur un serveur MySQL local (localhost, utilisateur root, mot de passe vide). La table `inscrits` doit contenir les champs : `id` (entier, clé primaire, auto-incrément), `nom` (varchar 50), `prenom` (varchar 50), `email` (varchar 100, unique), `date_inscription` (date), `montant_payé` (entier).

1. Écrire le code PHP complet de connexion à MySQL en utilisant l'orientation objet (`new mysqli(...)`) ou procédurale (`mysqli_connect`). Inclure la sélection de la base et la gestion d'erreur avec `die()` affichant Erreur de connexion : [message d'erreur].
2. Écrire la requête SQL de création de la table `inscrits` (instruction `CREATE TABLE IF NOT EXISTS`) et le code PHP pour l'exécuter via `mysqli_query`. Vérifier le succès de l'exécution.
3. Fermer proprement la connexion.
4. Expliquer pourquoi il est préférable de définir le jeu de caractères (`SET NAMES utf8mb4`) immédiatement après la connexion.

EXERCICE 3 Insertion et transaction – Gestion des erreurs

Reprenons la base `concours_yaounde`. On veut inscrire trois candidats dont les données sont stockées dans un tableau associatif PHP `$candidats` :

```
$candidats = [
    ['nom'=>'Fouda', 'prenom'=>'Jean', 'email'=>'jean.fouda@email.cm', 'date'=>'2025-01-15', 'montant'=>15000],
    ['nom'=>'Mballa', 'prenom'=>'Grace', 'email'=>'grace.mballa@email.cm', 'date'=>'2025-01-16', 'montant'=>12000],
    ['nom'=>'Essomba', 'prenom'=>'Paul', 'email'=>'paul.essomba@email.cm', 'date'=>'2025-01-17', 'montant'=>10000]
];
```

1. Écrire une boucle `foreach` parcourant `$candidats`. À l'intérieur, préparer une requête d'insertion paramétrée (`prepare`, `bind_param`, `execute`) pour insérer chaque candidat. Utiliser des marqueurs ?.
2. Gérer l'erreur potentielle de clé dupliquée (email unique) : si `execute()` échoue, afficher Échec pour [email] : [erreur] sans arrêter le script pour les suivants.
3. Compter et afficher le nombre total d'enregistrements réellement insérés (`affected_rows` cumulé).
4. Quelle est la différence fondamentale entre `mysqli_query` et la méthode `prepare/execute` concernant la sécurité (injection SQL) ?

EXERCICE 4 Sélection et affichage – Requêtes complexes

On souhaite afficher la liste des inscrits ayant payé plus de 10 000 FCFA, triés par date d'inscription décroissante. On veut afficher : Nom complet (Majuscules), Email, Date formatée JJ/MM/AAAA, Montant avec séparateur de milliers (ex : 15 000).

1. Écrire la requête SQL `SELECT` correspondante (utiliser `UPPER`, `CONCAT`, `DATE_FORMAT` côté SQL si possible, sinon côté PHP).

2. Écrire le code PHP pour exécuter la requête, récupérer les résultats sous forme de tableau associatif (`fetch_assoc`) dans une boucle `while`, et générer un tableau HTML (`<table>`) avec une ligne d'en-tête et une ligne par inscrit.
3. Modifier la requête pour n'afficher que les 2 premiers résultats (`LIMIT`).
4. Si la table contient 5000 lignes, pourquoi l'utilisation de `fetch_assoc` dans une boucle est-elle préférable à `fetch_all` pour l'affichage paginé ?

EXERCICE 5 Mise à jour et suppression – Cas pratique

La direction décide d'annuler l'inscription de tous les candidats n'ayant pas payé le montant exact correspondant à leur catégorie (voir grille Exercice 1). On ajoute une colonne `statut` (`varchar 20`, défaut `'valide'`) à la table `inscrits`.

1. Écrire la requête `ALTER TABLE` pour ajouter la colonne `statut`.
2. Proposer un algorithme (pseudo-code ou PHP structuré) pour : sélectionner tous les inscrits, recalculer le tarif théorique pour chacun (selon âge, catégorie déduite du montant payé ? Non, on suppose qu'on a stocké la catégorie. Ajoutons `categorie` dans la table pour cet exercice), et mettre à jour `statut` à `'annule'` si `montant_payé != tarif_theorique`.
3. Écrire la requête `UPDATE` paramétrée pour modifier le statut d'un `id` donné.
4. Écrire la requête `DELETE` pour supprimer définitivement les inscriptions ayant `statut = 'annule'` et antérieures au 01/01/2025. Préciser la clause `WHERE` exacte.